



Laboratório 7 - Visão Computacional
Detecção de Bordas com Algoritmo de Canny

Prof. Dr. Fagner de Assis Moura Pimentel
Inteligência Artificial e Robótica - CC7711

Grupo:

Deise Adriana Silva Araújo
Lucas Rebouças Silva

22.222.024-6
22.122.048-6

1 Introdução

A detecção de bordas é uma técnica fundamental em visão computacional, utilizada para identificar descontinuidades nas intensidades dos pixels de uma imagem.

2 Metodologia

2.1 Processamento da Imagem

O processamento foi realizado seguindo as seguintes etapas:

1. **Carregamento da imagem:** Importação da imagem no formato BGR (padrão OpenCV) e conversão para RGB
2. **Conversão para escala de cinza:** O algoritmo de Canny opera sobre imagens em escala de cinza
3. **Suavização:** Aplicação de filtro de média para redução de ruído
4. **Detecção de bordas:** Aplicação do algoritmo de Canny com thresholds calculados
5. **Sobreposição:** Marcação das bordas em vermelho na imagem original

2.2 Algoritmo de Canny

O algoritmo de Canny utiliza dois limiares (thresholds) para detecção de bordas:

- **threshold1:** Limiar inferior - bordas com gradiente abaixo deste valor são descartadas
- **threshold2:** Limiar superior - bordas com gradiente acima deste valor são aceitas
- Bordas com valores intermediários são aceitas apenas se conectadas a bordas fortes

2.3 Parâmetros Utilizados

- Tamanho do kernel de suavização: 5×5
- $\text{threshold1} = \frac{a}{2}$, onde a é o valor máximo de intensidade
- $\text{threshold2} = \frac{a}{2}$

3 Implementação

3.1 Código Python

```
1 import numpy as np
2 import cv2
3 import matplotlib.pyplot as plt
4
5 # Carregar a imagem
6 img = cv2.imread('./imagens/FEI01.jpg')
7 img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)
8
9 # Converter para escala de cinza
10 img_gray = cv2.cvtColor(img_rgb, cv2.COLOR_RGB2GRAY)
11
12 # Aplicar filtro de suavizacao
13 kernel_size = 5
14 img_blur = cv2.blur(img_gray, ksize=(kernel_size, kernel_size))
15
16 # Obtencao do valor maximo
17 a = img_blur.max()
18
19 # Deteccao de bordas com Canny
20 threshold1 = a / 2
21 threshold2 = a / 2
22 edges = cv2.Canny(image=img_blur,
23                   threshold1=threshold1,
24                   threshold2=threshold2)
25
26 # Criar imagem de saida com bordas em vermelho
27 img_output = img_rgb.copy()
28 img_output[edges == 255] = [255, 0, 0]
29
30 # Estatisticas
31 print(f"Pixels de borda: {np.sum(edges == 255)}")
32 print(f"Percentual: {100 * np.sum(edges == 255) / edges.size:.2f}%")
```

4 Resultados

4.1 Imagens Processadas

4.1.1 FEI01 - Prédio da FEI



Figura 1: FEI01 - Processo de detecção de bordas: (superior esquerda) Imagem original, (superior direita) Escala de cinza, (inferior esquerda) Bordas detectadas, (inferior direita) Resultado final com bordas em vermelho



Figura 2: FEI01 - Comparação direta: Imagem original vs. Bordas detectadas em vermelho

4.1.2 FEI02 - Vista Externa do Campus



Figura 3: FEI02 - Processo de detecção de bordas: (superior esquerda) Imagem original, (superior direita) Escala de cinza, (inferior esquerda) Bordas detectadas, (inferior direita) Resultado final com bordas em vermelho



Figura 4: FEI02 - Comparação direta: Imagem original vs. Bordas detectadas em vermelho

4.2 Estatísticas da Detecção

O processamento das imagens gerou as seguintes métricas comparativas:

Métrica	FEI01	FEI02
Pixels de borda detectados	19.638	68.583
Percentual de bordas (%)	3,83	3,31
Total de pixels na imagem	513.000	2.073.600
Dimensões (largura $\times$ altura)	1140 $\times$ 450	1920 $\times$ 1080

Tabela 1: Estatísticas comparativas da detecção de bordas nas imagens FEI01 e FEI02

4.3 Análise dos Resultados

Os resultados obtidos demonstram que o algoritmo de Canny foi eficaz na detecção das principais características estruturais das imagens, incluindo:

- Contornos dos edifícios
- Estruturas da torre e elementos arquitetônicos
- Elementos da vegetação
- Limites entre diferentes regiões das imagens
- Detalhes de janelas, portas e outras estruturas

A suavização prévia com filtro de média foi importante para reduzir ruídos e evitar a detecção de bordas espúrias. A escolha dos thresholds em $\frac{a}{2}$ mostrou-se adequada para balancear a sensibilidade do detector.

5 Espaço de Cores

O processamento de imagens em visão computacional frequentemente envolve diferentes espaços de cores:

- **RGB**: Espaço padrão, representando cores através de componentes Vermelho, Verde e Azul
- **Grayscale**: Reduz a imagem a um único canal de intensidade, essencial para o algoritmo de Canny
- **HSV**: Representa cores através de Matiz, Saturação e Valor
- **HLS**: Representa cores através de Matiz, Luminosidade e Saturação

Para detecção de bordas, a conversão para escala de cinza é essencial, pois o algoritmo de Canny opera sobre gradientes de intensidade luminosa.

6 Conclusão

O laboratório permitiu a aplicação prática de técnicas fundamentais de visão computacional, especialmente a detecção de bordas usando o algoritmo de Canny. Os resultados demonstram a eficácia do método na identificação de características estruturais relevantes em imagens de diferentes resoluções.

A escolha adequada dos parâmetros, especialmente os thresholds ($\frac{a}{2}$) e o tamanho do kernel de suavização (5×5), mostrou-se crucial para obter resultados satisfatórios e consistentes. O pré-processamento com filtro de média foi importante para melhorar a qualidade da detecção, reduzindo a influência de ruído.